

Módulo 4 – E-commerce CLI

Llegó el momento de poner en práctica lo aprendido en Java. Estos fundamentos serán el cimiento de tu backend: codifica con claridad, valida tus ideas y avanza con confianza. ¡Manos al código!

1) Propósito

Desarrollar una aplicación de **consola en Java** con dos flujos (Administrador y Usuario) para gestionar catálogo, carrito y compra, practicando **POO, control de flujo, colecciones, validaciones/excepciones y pruebas unitarias**.

2) Qué debe hacer la app:

Menú principal

- 1) ADMIN
- 2) USUARIO
- 0) Salir

ADMIN (gestión de productos)

- Listar productos
- Buscar (nombre/categoría)
- Crear producto (id único, nombre, categoría, precio > 0)
- Editar producto (nombre/categoría/precio)
- Eliminar producto (con confirmación)

USUARIO (carrito, descuento y compra)

- Listar / Buscar productos
- Agregar al carrito (id, cantidad > 0)
- Quitar del carrito (id)
- Ver carrito (items, subtotales, TOTAL base)
- **Ver descuentos activos** (mostrar reglas y su condición)
- Confirmar compra
 - Calcular **TOTAL base** y aplicar **automáticamente** todas las **reglas de descuento** que correspondan.
 - Mostrar **detalle de descuentos aplicados** y **TOTAL final**.
 - Crear la **Orden** en memoria y vaciar el carrito.

Validaciones obligatorias: id existente; cantidad entera > 0; no confirmar con carrito vacío.

Cálculos:

subtotal = precio × cantidad; totalBase = Σ subtotales; totalFinal = totalBase – descuentosAplicados.

Excepción personalizada: por ejemplo, CantidadInvalidaException.

3) Descuentos automáticos

Definir al menos **2** reglas y evaluarlas **sin intervención del usuario** al confirmar, por ejemplo:

- **DescuentoPorMonto(min, porcentaje)** → si `totalBase` $\geq$ min, descuenta %.
- **DescuentoPorCategoria(categoria, porcentaje)** → si el carrito contiene esa categoría, descuenta % sobre `totalBase`.

El usuario debe poder listar los descuentos vigentes por consola.

4) Requisitos técnicos mínimos

- Clases sugeridas: Producto, Carrito, Orden, Catalogo (gestión de productos), TiendaService (lógica), Consola (UI).
- Colecciones: List/Map; ordenamientos con Comparator (por nombre/precio).
- Tests (JUnit): ≥ 3 (totales de carrito; validación de cantidad; aplicación de descuentos).
- Datos en memoria.
- **Git/GitHub**: repo público ecommerce-cli-m4, **mínimo 3 commits** con mensajes descriptivos.

5) Entregables

- Proyecto comprimido en un único archivo .zip.
- README.md con enlace al repositorio público de GitHub, instrucciones de ejecución, menús y ejemplo breve de compra (incluye vista de descuentos activos y confirmación).

Rúbrica de evaluación

Criterio	Excelente (3 pts)	Adecuado (2 pts)	Básico (1 pt)	Insuficiente (0 pts)
POO y diseño	Clases con responsabilidades claras; bajo acoplamiento.	Correcto con leves acoplamientos.	Diseño confuso.	Sin estructura OO clara.
Flujos por rol	Menús de Admin y Usuario completos y estables.	Funciona con detalles menores.	Falta operación clave.	No se distinguen roles.
Colecciones y totales	Búsqueda/ordenamiento correctos; totales exactos.	Pequeños errores.	Inconsistencias.	Falla manejo de carrito/total.
Validaciones y excepciones	Entradas y cantidad (>0) bien tratadas; mensajes claros.	Aceptable con huecos.	Validaciones mínimas.	Sin manejo de errores.
Descuentos automáticos	Descuentos aplicados correctamente al confirmar, mostrando detalle.	Descuentos aplicados con errores menores.	Implementación parcial.	Sin descuentos automáticos.
Tests + Repo y commits	≥3 tests relevantes pasando; repo limpio; README claro con enlace.	2 tests útiles y README aceptable.	1 test o README escueto.	Sin tests o sin README.